

20/06/26

Class Test :-

Section A :-

Q.1. Ans

<u>List</u>	<u>Tuple</u>
Mutable	Immutable
Created using []	Created using ()
Method like append(), remove(), sort().	Fewer methods
Uses more memory	Uses less memory

Q2: Ans Mutable $\rightarrow$ Objects whose value changed after the creation.

Ex:- List, Dictionary, Set,

Immutable $\rightarrow$ Objects whose value can not changed after creation.

Ex:- Tuple, String, Integer, Float.

Q3: Ans Iterable $\rightarrow$ An object that can be looped over.

Ex:- List, Tuple, String.

Iterator: - An object that returns elements one by one using `next()`.
Created using `iter()`.

Q.4 Ans → Generator is a special function that uses the `yield` keyword to return value one at a time instead of returning all values at once.

Uses: -

- ①. Save memory
- ②. Useful for large datasets.
- ③. Generate values only when needed.

Q.5 ms → A Decorator is a function that adds extra functionality to another function without changing its original code.

Use Cases: -

①. Logging

②. Authentication

③. Access Control

④. Timing functions.

"9/11"

Q6 `nums = [10, 20, 30, 40, 50]`

`largest = nums[0]`

`second_larg = nums[0]`

`for i in nums:`

`if i > largest:`

`second_larg = largest`

`largest = i`

`elif i > second_larg and i != largest:`
`second_larg = i`

`print("second largest:", second_larg)`

Output → Second largest: 40

Q7 `s = input("Enter a string = ")`

`d = {}`

`for ch in s:`

`if ch in d:`

`d[ch] = d[ch] + 1`

`else:`

`d[ch] = 1`

`print(d)`

Output:-

Enter a string = hello

`{ 'h': 1, 'e': 1, 'l': 2, 'o': 1 }`

"Not"

Q8. s1 = input("Enter first string = ")
s2 = input("Enter second string = ")

if len(s1) != len(s2):
 print("Not Anagram")

else:
 count = {}

for ch in s1:
 if ch in count:
 count[ch] += 1
 else:
 count[ch] = 1

for ch in s2:
 if ch in count:
 count[ch] -= 1
 else:
 print("Not Anagram")
 break

else:
 flag = True

for value in count.values():
 if value != 0:
 flag = False
 break

if flag:
 print("Anagram")

"977"

else:

print("Not Anagram")

Q9. class Employee:

```
def __init__(self, name, salary):  
    self.name = name  
    self.salary = salary
```

```
def display(self):  
    print("Name:", self.name)  
    print("Salary:", self.salary)
```

e1 = Employee("Dushyant", 50000)
e1.display()

Output: - Name: Dushyant
Salary: 50000

Q10. def fibonacci(n):
 a = 0
 b = 1

```
for i in range(n):  
    yield a  
    a, b = b, a + b
```

n = int(input("Enter N: "))

"971"

```
for num in fibonacci(n):  
    print(num)
```

Output: - Enter N: 7

0

1

1

2

3

5

8